22. JDBC Examples

Example1 for Creating a Database:

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCExample1 {
    static final String DB_URL = "jdbc:mysql://localhost/";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        try {
            // Open a connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            stmt = conn.createStatement();

            // Execute a query
            String sql = "CREATE DATABASE STUDENTS";
            stmt.executeUpdate(sql);
            System.out.println("Database created successfully...");
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close resources
            try {
                if (stmt != null) {
                    stmt.close();
                }
                if (conn != null) {
                    conn.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

Output:

```
Database created successfully...
```

Example2 for Creating a Schema:

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCExample2 {
    static final String DB_URL = "jdbc:mysql://localhost/";
    static final String USER = "root";
```

```
    static final String PASS = "root";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        try {
            // Open a connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            stmt = conn.createStatement();

            // Execute a query
            String sql = "CREATE SCHEMA Sample_db1";
            stmt.executeUpdate(sql);
            System.out.println("Schema created successfully...");
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close resources
            try {
                if (stmt != null) {
                    stmt.close();
                }
                if (conn != null) {
                    conn.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

Output:

```
Schema created successfully...
```

Example for Selecting a Database:

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class JDBCExample3 {
    static final String DB_URL = "jdbc:mysql://localhost/organization";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String[] args) {
        System.out.println("Connecting to a selected database...");
        Connection conn = null;
        try {
            // Open a connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            System.out.println("Connected database successfully...");
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close the connection
            try {
                if (conn != null) {
                    conn.close();
```

```
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

Output:

```
Connecting to a selected database...
Connected database successfully...
```

Example4 for Getting Records from a Table of Selected Database:

```
package com.lokesh;

import java.sql.*;

// This class demonstrates use of selecting a database.
public class JDBCExample4 {
    static final String DB_URL = "jdbc:mysql://localhost/";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs1 = null;
        ResultSet rs2 = null;

        try {
            // Open a connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            System.out.println("Connected database successfully...");

            // Create a statement
            stmt = conn.createStatement();

            // Query to show databases
            rs1 = stmt.executeQuery("SHOW DATABASES");
            System.out.println("DATABASES");
            System.out.println("-----------------------------------------");
            while (rs1.next()) {
                System.out.println(rs1.getString(1));
            }

            System.out.println("-------------------------------------------------------");

            // Select the database TUTORIALSPOINT
            stmt.executeUpdate("USE ORGANIZATION");

            // Query to select data from employees table
            rs2 = stmt.executeQuery("SELECT * FROM employees");
            System.out.println("Id of employees");
            while (rs2.next()) {
                System.out.println("id= " + rs2.getInt("id"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close ResultSet rs2
            try {
```

```
                if (rs2 != null) {
                    rs2.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }

            // Close ResultSet rs1
            try {
                if (rs1 != null) {
                    rs1.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }

            // Close Statement
            try {
                if (stmt != null) {
                    stmt.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }

            // Close Connection
            try {
                if (conn != null) {
                    conn.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

Output:

```
Connected database successfully...
DATABASES
-----------------------------------------
first
ineuron
information_schema
mysql
organization
performance_schema
sakila
sample_db1
second
students
sys
world
-------------------------------------------------------
Id of employees
id= 1
id= 2
id= 3
id= 4
id= 5
id= 7
```

Example5 for Getting Current Database and Table Names of Selected Database:

```
package com.lokesh;

import java.sql.*;

//This class demonstrates use of SELECT DATABASE() command and SHOW TABLES
public class JDBCExample5 {

 static final String DB_URL = "jdbc:mysql://localhost/";
 static final String USER = "root";
 static final String PASS = "root";

 public static void main(String[] args) {
     Connection conn = null;
     Statement stmt = null;
     ResultSet rs1 = null;
     ResultSet rs2 = null;

     try {
         // Open a connection
         conn = DriverManager.getConnection(DB_URL, USER, PASS);
         System.out.println("Connected database successfully...");

         // Create a statement
         stmt = conn.createStatement();

         // Make TUTORIALSPOINT the current database
         stmt.executeUpdate("USE ORGANIZATION");

         // Check the currently selected database
         rs1 = stmt.executeQuery("SELECT DATABASE()");
         while (rs1.next()) {
             System.out.println("Current database: " + rs1.getString(1));
         }

         // Show tables in the current database
         rs2 = stmt.executeQuery("SHOW TABLES");
         System.out.println("List of tables in current database TUTORIALSPOINT");
         System.out.println("-------------------------------------------------");
         while (rs2.next()) {
             System.out.println(rs2.getString(1));
         }

     } catch (SQLException e) {
         e.printStackTrace();
     } finally {
         // Close ResultSet rs2
         try {
             if (rs2 != null) {
                 rs2.close();
             }
         } catch (SQLException se) {
             se.printStackTrace();
         }

         // Close ResultSet rs1
         try {
             if (rs1 != null) {
                 rs1.close();
             }
         } catch (SQLException se) {
             se.printStackTrace();
         }

         // Close Statement
```

```
         try {
             if (stmt != null) {
                 stmt.close();
             }
         } catch (SQLException se) {
             se.printStackTrace();
         }

         // Close Connection
         try {
             if (conn != null) {
                 conn.close();
             }
         } catch (SQLException se) {
             se.printStackTrace();
         }
     }
 }
}
```

Output:

```
Connected database successfully...
Current database: organization
List of tables in current database TUTORIALSPOINT
-------------------------------------------------
blobexample
clobexample
company
datetimeexample
employee
employees
```

Example6 for Dropping a Database:

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCExample6 {
    static final String DB_URL = "jdbc:mysql://localhost/";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;

        try {
            // Open a connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            stmt = conn.createStatement();

            // Execute SQL to drop the database
            String sql = "DROP DATABASE sample_db1";
            stmt.executeUpdate(sql);
            System.out.println("Database dropped successfully...");
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
```

```
            // Close Statement
            try {
                if (stmt != null) {
                    stmt.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }

            // Close Connection
            try {
                if (conn != null) {
                    conn.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

Output:

```
Database dropped successfully...
```

Example7 for Creating a Table:

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCExample7 {
    static final String DB_URL = "jdbc:mysql://localhost/organization";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;

        try {
            // Open a connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            stmt = conn.createStatement();

            // SQL query to create a table
            String sql = "CREATE TABLE REGISTRATION " +
                         "(id INTEGER not NULL, " +
                         " first VARCHAR(255), " +
                         " last VARCHAR(255), " +
                         " age INTEGER, " +
                         " PRIMARY KEY ( id ))";

            stmt.executeUpdate(sql);
            System.out.println("Created table in given database...");
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close Statement
            try {
```

```
                if (stmt != null) {
                    stmt.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }

            // Close Connection
            try {
                if (conn != null) {
                    conn.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

Output:

```
Created table in given database...
```

Example8 for Creating a Temporary Table:

```
package com.lokesh;

import java.sql.*;

public class JDBCExample8 {

   static final String DB_URL = "jdbc:mysql://localhost/organization";
   static final String USER = "root";
   static final String PASS = "root";

   public static void main(String args[]) {
      try{
         Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
         Statement stmt = conn.createStatement();

         String QUERY1 = "CREATE TEMPORARY TABLE EMPLOYEES_COPY SELECT * FROM EMPLOYEES";
         stmt.execute(QUERY1);
         String QUERY2 = "SELECT * FROM EMPLOYEES_COPY";
         ResultSet rs = stmt.executeQuery(QUERY2);

         while (rs.next()){
            System.out.print("Id: " + rs.getInt("id"));
            System.out.print(" Age: " + rs.getInt("age"));
            System.out.print(" First: " + rs.getString("first"));
            System.out.println(" Last: " + rs.getString("last"));
            System.out.println("----------------------------------------");
         }
      }catch (SQLException e){
         e.printStackTrace();
      }
   }
}
```

Output:

```
Id: 1 Age: 20 First: Rita Last: Tez
----------------------------------------
Id: 2 Age: 20 First: Sita Last: Singh
```

```
----------------------------------------
Id: 3 Age: 30 First: Zaid Last: Khan
----------------------------------------
Id: 4 Age: 33 First: Sumit Last: Mittal
----------------------------------------
Id: 5 Age: 40 First: John Last: Paul
----------------------------------------
Id: 7 Age: 20 First: Sita Last: Singh
----------------------------------------
```

Example9 for Creating a Duplicate Table:

```
package com.lokesh;

import java.sql.*;

//This class demonstrates the way of creating a table which is exactly similar to another
table.
public class JDBCExample9 {

	static final String DB_URL = "jdbc:mysql://localhost/organization";
	static final String USER = "root";
	static final String PASS = "root";

	public static void main(String args[]) {

		try {
			Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
			Statement stmt = conn.createStatement();
			String QUERY1 = "CREATE TABLE EMPLOYEES_O LIKE EMPLOYEES";
			stmt.execute(QUERY1);

			System.out.println("Duplicate Table Created");

		} catch (SQLException e) {
			e.printStackTrace();
		}
	}
}
```

Output:

```
Duplicate Table Created
```

Example10 for Dropping a Table:

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCExample10 {
    static final String DB_URL = "jdbc:mysql://localhost/organization";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
```

```
        try {
            // Open a connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            stmt = conn.createStatement();

            // SQL query to drop the table
            String sql = "DROP TABLE REGISTRATION";
            stmt.executeUpdate(sql);
            System.out.println("Table deleted in given database...");
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close Statement
            try {
                if (stmt != null) {
                    stmt.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }

            // Close Connection
            try {
                if (conn != null) {
                    conn.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

Output:

```
Table deleted in given database...
```

Example11 for Dropping a Table if Exists:

```
package com.lokesh;

import java.sql.*;

// This file demonstrates use of DROP TABLE IF EXISTS command
public class JDBCExample11 {

   static final String DB_URL = "jdbc:mysql://localhost/organization";
   static final String USER = "root";
   static final String PASS = "root";

   public static void main(String args[]) {
      try{
         Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
         Statement stmt = conn.createStatement();
         String QUERY = "DROP TABLE IF EXISTS emptest";
         stmt.execute(QUERY);

         System.out.println("Table emptest dropped successfully.");
         System.out.println("----------------------------------------");
         ResultSet rs = stmt.executeQuery("show tables");
         System.out.println("List of tables");
         System.out.println("----------------------------------------");
         while(rs.next()){
```

```
            System.out.println(rs.getString(1));
         }
      }catch (SQLException e){
         e.printStackTrace();
      }
   }
}
```

Output:

```
Table emptest dropped successfully.
---------------------------------------
List of tables
---------------------------------------
blobexample
clobexample
company
datetimeexample
deptest
employee
employees
employees_o
```

Example12 for Truncate Table:

```
package com.lokesh;

import java.sql.*;

public class JDBCExample12 {

   static final String DB_URL = "jdbc:mysql://localhost/organization";
   static final String USER = "root";
   static final String PASS = "root";

   public static void main(String args[]) {

      try {
         Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
         Statement stmt = conn.createStatement();
         String sel_query = "select * from deptest";
         ResultSet rs1 = stmt.executeQuery(sel_query);
         System.out.println("Rows from deptest");
         System.out.println("--------------------");
         while(rs1.next()) {
            System.out.println("dept_id: " + rs1.getInt(1));
            System.out.println("dept_name: " + rs1.getString(2));
         }

         String QUERY = "TRUNCATE TABLE deptest";
         stmt.execute(QUERY);

         System.out.println("Table deptest rows successfully deleted.");
         System.out.println("----------------------------------------");
         System.out.println(" Doing a select on deptest...");
         rs1 = stmt.executeQuery(sel_query);
         if (!rs1.next()) {
            System.out.println(" **** ResultSet is empty *****");
         }
      } catch(SQLException e){
         e.printStackTrace();
      }
   }
```

```
}
```

Output:

```
Rows from deptest
--------------------
dept_id: 1
dept_name: Human Resources
dept_id: 2
dept_name: Finance
dept_id: 3
dept_name: IT
dept_id: 4
dept_name: Marketing
dept_id: 5
dept_name: Sales
Table deptest rows successfully deleted.
---------------------------------------
 Doing a select on deptest...
 **** ResultSet is empty *****
```

Example13 for Inserting Record in a Table:

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCExample13 {
	static final String DB_URL = "jdbc:mysql://localhost/organization";
	static final String USER = "root";
	static final String PASS = "root";

	public static void main(String[] args) {
		Connection conn = null;
		Statement stmt = null;

		try {
			// Open a connection
			conn = DriverManager.getConnection(DB_URL, USER, PASS);
			stmt = conn.createStatement();

			// Execute a query
			System.out.println("Inserting records into the table...");
			String sql = "INSERT INTO Registration VALUES (100, 'Zara', 'Ali',
                                                                    18)";
			stmt.executeUpdate(sql);
			sql = "INSERT INTO Registration VALUES (101, 'Mahnaz', 'Fatma', 25)";
			stmt.executeUpdate(sql);
			sql = "INSERT INTO Registration VALUES (102, 'Zaid', 'Khan', 30)";
			stmt.executeUpdate(sql);
			sql = "INSERT INTO Registration VALUES (103, 'Sumit', 'Mittal', 28)";
			stmt.executeUpdate(sql);
			System.out.println("Inserted records into the table...");
		} catch (SQLException e) {
			e.printStackTrace();
		} finally {
			// Close Statement
			try {
				if (stmt != null) {
					stmt.close();
```

```
                        }
                    } catch (SQLException se) {
                        se.printStackTrace();
                    }

                    // Close Connection
                    try {
                        if (conn != null) {
                            conn.close();
                        }
                    } catch (SQLException se) {
                        se.printStackTrace();
                    }
                }
            }
}
```

Output:

```
Inserting records into the table...
Inserted records into the table...
```

Example14 for Inserting Record in Single Statement in a Table:

```
package com.lokesh;

import java.sql.*;

// This class demonstrates use of multiple inserts within a single SQL
public class JDBCExample14 {

   static final String DB_URL = "jdbc:mysql://localhost/organization";
   static final String USER = "root";
   static final String PASS = "root";

   public static void main(String args[]) {
      try {
         Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
         Statement stmt = conn.createStatement();
         stmt.execute("INSERT INTO sampletable(id, name) VALUES(3, 'Sachin'), (4,
                                                                    'Kishore')");
         System.out.println("----- Successfully inserted into table sampletable ----\n\n");
         System.out.println("Displaying records from sampletable table, showing inserted
                                                                        values");
         System.out.println("----------------------------");
         ResultSet rs = stmt.executeQuery("select * from sampletable");

         while(rs.next()){
            System.out.println("id: " + rs.getInt(1));
            System.out.println("name: " + rs.getString(2));
         }
      } catch(SQLException e) {
         e.printStackTrace();
      }
   }
}
```

Output:

```
----- Successfully inserted into table sampletable ----


Displaying records from sampletable table, showing inserted values
```

```
--------------------------
id: 3
name: Sachin
id: 4
name: Kishore
```

Example15 for Inserting Record Using Select Statement in a Table:

```
package com.lokesh;

import java.sql.*;

//This class demonstrates use of INSERT..SELECT SQL,
//where data is inserted in table using select from another table.
public class JDBCExample15 {
	static final String DB_URL = "jdbc:mysql://localhost/organization";
	static final String USER = "root";
	static final String PASS = "root";

	public static void main(String args[]) {

		try {
			Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
			Statement stmt = conn.createStatement();

			// Data from students table (student id, first name) is inserted into
			// sampledb4(id, name)
			String ins_sel = "insert into sampletable(id, name) select
                                                                studentid,"
                                + " firstname from students where studentid > 1004";

			stmt.executeUpdate(ins_sel);

			ResultSet rs = stmt.executeQuery("select * from sampletable ");

			System.out.println("Displaying records of table sampletable/ Ids" +
                                        " greater than 1004 are from students table");
			System.out.println("-------------------------------------");

			while (rs.next()) {
				System.out.print("id: " + rs.getInt(1));
				System.out.println(" name: " + rs.getString(2));
			}
		} catch (SQLException e) {
			e.printStackTrace();
		}
	}
}
```

Output:

```
Displaying records of table sampledb4/ Ids greater than 1004 are from students table
-----------------------------------
id: 3 name: Sachin
id: 4 name: Kishore
id: 1005 name: Sarah
id: 1006 name: Sachin
```

Example16 for Selecting Record from a Table:

```
package com.lokesh;
```

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCExample16 {
    static final String DB_URL = "jdbc:mysql://localhost/organization";
    static final String USER = "root";
    static final String PASS = "root";
    static final String QUERY = "SELECT id, first, last, age FROM Registration";

    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;

        try {
            // Open a connection
            conn = DriverManager.getConnection(DB_URL, USER, PASS);

            // Execute a query
            stmt = conn.createStatement();
            rs = stmt.executeQuery(QUERY);

            // Process the ResultSet
            while (rs.next()) {
                // Display values
                System.out.print("ID: " + rs.getInt("id"));
                System.out.print(", Age: " + rs.getInt("age"));
                System.out.print(", First: " + rs.getString("first"));
                System.out.println(", Last: " + rs.getString("last"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close ResultSet
            try {
                if (rs != null) {
                    rs.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }

            // Close Statement
            try {
                if (stmt != null) {
                    stmt.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }

            // Close Connection
            try {
                if (conn != null) {
                    conn.close();
                }
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

Output:

```
ID: 100, Age: 18, First: Zara, Last: Ali
ID: 101, Age: 25, First: Mahnaz, Last: Fatma
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 103, Age: 28, First: Sumit, Last: Mittal
```

Example17 for Selecting Records from a Table with Order By:

```
package com.lokesh;

import java.sql.*;

// This class demonstrates use of ORDER BY clause of the SELECT statement in SQL
public class JDBCExample17 {

	static final String DB_URL = "jdbc:mysql://localhost/organization";
	static final String USER = "root";
	static final String PASS = "root";

	public static void main(String args[]) {
		try {
			Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
			Statement stmt = conn.createStatement();
			// Example of ORDER BY
			String query1 = "select id, age, first, last from employees order by
                                                                    age asc;";

			ResultSet rs = stmt.executeQuery(query1);
			while (rs.next()) {
				System.out.print(" ID: " + rs.getInt(1));
				System.out.print(" AGE: " + rs.getInt(2));
				System.out.print(" FirstName: " + rs.getString(3));
				System.out.println(" LastName: " + rs.getString(4));
			}

			rs.close();
			stmt.close();
			conn.close();
		} catch (SQLException e) {
			e.printStackTrace();
		}
	}
}
```

Output:

```
 ID: 1 AGE: 20 FirstName: Rita LastName: Tez
 ID: 2 AGE: 20 FirstName: Sita LastName: Singh
 ID: 7 AGE: 20 FirstName: Sita LastName: Singh
 ID: 3 AGE: 30 FirstName: Zaid LastName: Khan
 ID: 4 AGE: 33 FirstName: Sumit LastName: Mittal
 ID: 5 AGE: 40 FirstName: John LastName: Paul
```

Example18 Updating Record in a Table Using Statement Object:

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
```

```
import java.sql.SQLException;
import java.sql.Statement;

public class JDBCExample18 {
	static final String DB_URL = "jdbc:mysql://localhost/organization";
	static final String USER = "root";
	static final String PASS = "root";
	static final String QUERY = "SELECT id, first, last, age FROM Registration";

	public static void main(String[] args) {
		Connection conn = null;
		Statement stmt = null;
		ResultSet rs = null;

		try {
			// Open a connection
			conn = DriverManager.getConnection(DB_URL, USER, PASS);
			stmt = conn.createStatement();

			// Execute update query
			String sql = "UPDATE Registration SET age = 30 WHERE id in (100,
                                                                        101)";
			stmt.executeUpdate(sql);

			// Execute select query
			rs = stmt.executeQuery(QUERY);
			while (rs.next()) {
				// Display values
				System.out.print("ID: " + rs.getInt("id"));
				System.out.print(", Age: " + rs.getInt("age"));
				System.out.print(", First: " + rs.getString("first"));
				System.out.println(", Last: " + rs.getString("last"));
			}

		} catch (SQLException e) {
			e.printStackTrace();
		} finally {
			// Close resources in reverse order
			try {
				if (rs != null)
					rs.close();
				if (stmt != null)
					stmt.close();
				if (conn != null)
					conn.close();
			} catch (SQLException e) {
				e.printStackTrace();
			}
		}
	}
}
```

Output:

```
ID: 100, Age: 30, First: Zara, Last: Ali
ID: 101, Age: 30, First: Mahnaz, Last: Fatma
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 103, Age: 28, First: Sumit, Last: Mittal
```

Example19 Updating Record in a Table Using PreparedStatement:

```
package com.lokesh;
```

```
import java.sql.*;

// This class demonstrates use of UPDATE using Java PreparedStatement
public class JDBCExample19 {

	static final String DB_URL = "jdbc:mysql://localhost/organization";
	static final String USER = "root";
	static final String PASS = "root";

	public static void main(String args[]) {

		try {
			Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);

			String upd_qry = "update employees set age = ? where id =? ";
			PreparedStatement pstmt = conn.prepareStatement(upd_qry);
			pstmt.setInt(1, 36);
			pstmt.setInt(2, 3);
			pstmt.executeUpdate();

			String sel_qry = "select * from employees where id = 3";
			ResultSet rs = pstmt.executeQuery(sel_qry);

			System.out.println("Displaying updated record..");

			while (rs.next()) {
				System.out.print(" ID: " + rs.getInt(1));
				System.out.print(" FirstName: " + rs.getString(2));
				System.out.print(" LastName: " + rs.getString(3));
				System.out.println(" AGE: " + rs.getInt(4));
			}

			rs.close();
			pstmt.close();
			conn.close();

		} catch (SQLException e) {
			e.printStackTrace();
		}
	}
}
```

Output:

```
Displaying updated record..
 ID: 3 FirstName: Zaid LastName: Khan AGE: 36
```

Example20 Updating Multiple Columns in single SQL:

```
package com.lokesh;

import java.sql.*;

//This class demonstrates use of updating multiple columns with a single SQL command
public class JDBCExample20 {

	static final String DB_URL = "jdbc:mysql://localhost/organization";
	static final String USER = "root";
	static final String PASS = "root";

	public static void main(String args[]) {

		try {
```

```
                        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);

                        String upd_qry = "update employees set age = 50, first='Shahbaz'
                                                                        where id =1 ";
                        Statement stmt = conn.createStatement();
                        stmt.executeUpdate(upd_qry);

                        String sel_qry = "select * from employees where id = 1";
                        ResultSet rs = stmt.executeQuery(sel_qry);

                        System.out.println("Displaying updated record..");

                        while (rs.next()) {
                                System.out.print(" ID: " + rs.getInt(1));
                                System.out.print(", FirstName: " + rs.getString(2));
                                System.out.print(", LastName: " + rs.getString(3));
                                System.out.println(", AGE: " + rs.getInt(4));
                        }

                        rs.close();
                        stmt.close();
                        conn.close();

                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

Output:

```
Displaying updated record..
 ID: 1, FirstName: Shahbaz, LastName: Tez, AGE: 50
```

Example21 for Deleting record from a Table:

```
package com.lokesh;

import java.sql.*;

public class JDBCExample21 {

        static final String DB_URL = "jdbc:mysql://localhost/organization";
        static final String USER = "root";
        static final String PASS = "root";
        static final String QUERY = "SELECT id, first, last, age FROM Registration";

        public static void main(String args[]) {

                try {
                        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
                        String sql = "DELETE FROM Registration WHERE id = 101";
                        Statement stmt = conn.createStatement();
                        stmt.executeUpdate(sql);
                        ResultSet rs = stmt.executeQuery(QUERY);

                        while(rs.next()) {
                                //Display values
                                System.out.print("ID: " + rs.getInt("id"));
                                System.out.print(", Age: " + rs.getInt("age"));
                                System.out.print(", First: " + rs.getString("first"));
                                System.out.println(", Last: " + rs.getString("last"));
                        }
```

```
                        rs.close();

                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

Output:

```
ID: 100, Age: 30, First: Zara, Last: Ali
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 103, Age: 28, First: Sumit, Last: Mittal
```

Example22 for Deleting records using Limit from a Table:

```
package com.lokesh;

import java.sql.*;

public class JDBCExample22 {

        static final String DB_URL = "jdbc:mysql://localhost/organization";
        static final String USER = "root";
        static final String PASS = "root";

        public static void main(String args[]) {

                try {
                        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
                        String sel_qry = "select * from employees ";
                        String del_qry = "DELETE FROM employees ORDER BY age LIMIT 3 ";
                        Statement stmt = conn.createStatement();

                        ResultSet rs = stmt.executeQuery(sel_qry);
                        System.out.println(" Displaying records before deletion ");
                        System.out.println(" ---------------------------------" );
                        showResults(rs);

                        stmt.executeUpdate(del_qry);
                        System.out.println("Displaying records after deletion..");
                        System.out.println(" ---------------------------------" );
                        rs = stmt.executeQuery(sel_qry);
                        showResults(rs);
                        rs.close();
                        stmt.close();
                        conn.close();

                } catch (SQLException e) {
                        e.printStackTrace();
                }

        }

        public static void showResults(ResultSet res) {
                try {
                        while(res.next()) {
                                System.out.print("ID: " + res.getInt(1));
                                System.out.print(", FirstName: " + res.getString(2));
                                System.out.print(", LastName: " + res.getString(3));
                                System.out.println(", AGE: " + res.getInt(4));
                        }
```

```
                        System.out.println(" ---------------------------------" );
                } catch(SQLException sqle) {
                        sqle.printStackTrace();
                }
        }
}
```

Output:

```
Displaying records before deletion
 ---------------------------------
ID: 1, FirstName: Shahbaz, LastName: Tez, AGE: 50
ID: 2, FirstName: Aisha, LastName: Rao, AGE: 22
ID: 3, FirstName: Zaid, LastName: Khan, AGE: 36
ID: 4, FirstName: Karan, LastName: Mehta, AGE: 27
ID: 5, FirstName: John, LastName: Paul, AGE: 40
ID: 6, FirstName: Emily, LastName: Clark, AGE: 24
ID: 7, FirstName: Arjun, LastName: Verma, AGE: 30
ID: 8, FirstName: Sara, LastName: Ali, AGE: 26
 ---------------------------------
Displaying records after deletion..
 ---------------------------------
ID: 1, FirstName: Shahbaz, LastName: Tez, AGE: 50
ID: 3, FirstName: Zaid, LastName: Khan, AGE: 36
ID: 4, FirstName: Karan, LastName: Mehta, AGE: 27
ID: 5, FirstName: John, LastName: Paul, AGE: 40
ID: 7, FirstName: Arjun, LastName: Verma, AGE: 30
 ---------------------------------
```

Example 23 for Selecting Records from a Table on Given Condition:

```
package com.lokesh;

import java.sql.*;

public class JDBCExample23 {

        static final String DB_URL = "jdbc:mysql://localhost/organization";
        static final String USER = "root";
        static final String PASS = "root";
        static final String QUERY = "SELECT id, first, last, age FROM Registration";

        public static void main(String args[]) {

                try {
                        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
                        Statement stmt = conn.createStatement();

                        System.out.println("Fetching records without condition...");
                        ResultSet rs = stmt.executeQuery(QUERY);
                        while (rs.next()) {
                                // Display values
                                System.out.print("ID: " + rs.getInt("id"));
                                System.out.print(", Age: " + rs.getInt("age"));
                                System.out.print(", First: " + rs.getString("first"));
                                System.out.println(", Last: " + rs.getString("last"));
                        }

                        // Select all records having ID equal or greater than 101
                        System.out.println("Fetching records with condition...");
                        String sql = "SELECT id, first, last, age FROM Registration" +
                                     " WHERE id >= 101 ";
                        rs = stmt.executeQuery(sql);
```

```
                        while (rs.next()) {
                                // Display values
                                System.out.print("ID: " + rs.getInt("id"));
                                System.out.print(", Age: " + rs.getInt("age"));
                                System.out.print(", First: " + rs.getString("first"));
                                System.out.println(", Last: " + rs.getString("last"));
                        }
                        rs.close();
                        stmt.close();
                        conn.close();

                } catch (SQLException e) {
                        e.printStackTrace();
                }

        }

}
```

Output:

```
Fetching records without condition...
ID: 100, Age: 30, First: Zara, Last: Ali
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 103, Age: 28, First: Sumit, Last: Mittal
Fetching records with condition...
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 103, Age: 28, First: Sumit, Last: Mittal
```

Example for Selecting Record from a Table:

```
package com.lokesh;

import java.sql.*;

public class JDBCExample24 {

        static final String DB_URL = "jdbc:mysql://localhost/organization";
        static final String USER = "root";
        static final String PASS = "root";
        static final String QUERY = "SELECT id, first, last, age FROM Registration";

        public static void main(String args[]) {

                try {
                        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
                        Statement stmt = conn.createStatement();

                        System.out.println("Fetching records without condition...");
                        ResultSet rs = stmt.executeQuery(QUERY);
                        while (rs.next()) {
                                // Display values
                                System.out.print("ID: " + rs.getInt("id"));
                                System.out.print(", Age: " + rs.getInt("age"));
                                System.out.print(", First: " + rs.getString("first"));
                                System.out.println(", Last: " + rs.getString("last"));
                        }

                        // Select all records having ID equal or greater than 101
                        System.out.println("Fetching records with condition...");
                        String sql = "SELECT id, first, last, age FROM Registration" +
```

```
                                " WHERE first LIKE '%za%'";
                        rs = stmt.executeQuery(sql);

                        while (rs.next()) {
                                // Display values
                                System.out.print("ID: " + rs.getInt("id"));
                                System.out.print(", Age: " + rs.getInt("age"));
                                System.out.print(", First: " + rs.getString("first"));
                                System.out.println(", Last: " + rs.getString("last"));
                        }
                        rs.close();
                        stmt.close();
                        conn.close();

                } catch (SQLException e) {
                        e.printStackTrace();
                }

        }

}
```

Output:

```
Fetching records without condition...
ID: 100, Age: 30, First: Zara, Last: Ali
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 103, Age: 28, First: Sumit, Last: Mittal
Fetching records with condition...
ID: 100, Age: 30, First: Zara, Last: Ali
ID: 102, Age: 30, First: Zaid, Last: Khan
```

Example 25 for Sorting Records of a Table:

```
package com.lokesh;

import java.sql.*;

public class JDBCExample25 {

        static final String DB_URL = "jdbc:mysql://localhost/organization";
        static final String USER = "root";
        static final String PASS = "root";
        static final String QUERY = "SELECT id, first, last, age FROM Registration";

        public static void main(String args[]) {

                try {
                        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
                        Statement stmt = conn.createStatement();

                        System.out.println("Fetching records in ascending order...");
                        ResultSet rs = stmt.executeQuery(QUERY + " ORDER BY first ASC");

                        while (rs.next()) {
                                // Display values
                                System.out.print("ID: " + rs.getInt("id"));
                                System.out.print(", Age: " + rs.getInt("age"));
                                System.out.print(", First: " + rs.getString("first"));
                                System.out.println(", Last: " + rs.getString("last"));
                        }

                        System.out.println("Fetching records in descending order...");
```

```
                        rs = stmt.executeQuery(QUERY + " ORDER BY first DESC");

                        while (rs.next()) {
                                // Display values
                                System.out.print("ID: " + rs.getInt("id"));
                                System.out.print(", Age: " + rs.getInt("age"));
                                System.out.print(", First: " + rs.getString("first"));
                                System.out.println(", Last: " + rs.getString("last"));
                        }

                        rs.close();
                        stmt.close();
                        conn.close();

                } catch (SQLException e) {
                        e.printStackTrace();
                }

        }

}
```

Output:

```
Fetching records in ascending order...
ID: 103, Age: 28, First: Sumit, Last: Mittal
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 100, Age: 30, First: Zara, Last: Ali
Fetching records in descending order...
ID: 100, Age: 30, First: Zara, Last: Ali
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 103, Age: 28, First: Sumit, Last: Mittal
```